

BEIRUT ARAB UNIVERITY

FACULTY OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

CMPS441 PROJECT

Supervised by: Dr Mahmoud Al-Skafi

And

T.A. Lynn El Arwadi



Hospital Emergency Triage System

Nour Ajami – 202302485
Sadeq Sleiman – 202302840

CONTENTS

1. **Project Title & Description**
2. **Problem Statement**
3. **Input Description**
4. **Triage Level**
5. **System Architecture**
6. **Algorithm Used**
7. **Pivot Strategy Comparison**
8. **Program Features**
9. **Conclusion**
10. **Full Code**
11. **Sample Output**

1. Project Title & Description

This project presents a Hospital Emergency Triage System designed using algorithmic techniques.
Its main goal is to assist hospitals in prioritizing patients based on their severity state.

The system uses:

- **QuickSort:** To sort the entire patient list from the most critical to the least critical.
- **QuickSelect:** To identify the k-th most critical patient without it being sorted.
- **Two pivot strategies:** Randomized and Deterministic to compare performance analysis.

2. Problem Statement

Emergency departments commonly face a considerable number of patients arriving at once. It is essential for doctors to identify who needs immediate care. Medical teams must quickly answer these questions:

- Who is the most critical?
- Who should we see next?
- How can we classify patients by urgency?
- Can we quickly locate the k-th most critical patient?

Manual sorting is a slow process that can lead to delay, particularly where there is a large patient count.

Our system solves these problems by providing a fast, algorithm-based solutions:

- Sorting patients by severity using QuickSort.
- Allowing staff to enter data manually or generate random test case.
- Providing useful triage and statistical insights.
- Quickly identify the k-th most critical patient.

This ensures hospitals treat patients based on medical priority.

3. Input Description

The system supports two types of inputs:

1. Manual Input:

   The user provides the name and severity for every patient.

2. Randomly Generated Data:

   The system generated n patients with:

   - Automatic names (Patient_1, Patient_2, …)
   - Random severity values.

4. Triage Levels

Severity	Level	Meaning
Greater than 75	Level 1	Critical
50 - 75	Level 2	High priority
25-49	Level 3	Normal priority
Less than 25	Level 4	Minor

5. System Architecture

1. Patient
   Stores:
   - Name
   - Severity score
2. QuickAlgorithms

   Implements:

   - QuickSort
   - QuickSelect
   - Median of three pivot
   - Random pivot
   - Partition and swap helpers

3. TriageSystem (main)

   Handles:

   - System menu
   - Input generation
   - Sorting & selecting
   - Statistics
   - Triage help
   - Runtime measurement
   - Algorithm comparison

6. Algorithm Used

1. QuickSort
   This method is used to sort the full list of patients from the critical to the least critical.

   QuickSort Complexities:
   - Best case: O (n log n)
   - Average case: O (n log n)
   - Worst case: O (n^2)

2. QuickSelect

   This method is used to determine the k-th most critical patient without sorting the entire list.

   QuickSelect Complexities:
   - Best case: O (n)
   - Average case: O (n)
   - Worst case: O (n^2)

7. Pivot Strategy

The system allows the user to choose:

1. Randomized Pivot
    - Pivot chosen randomly.
    - Avoids worst case.
    - Often faster on average.
2. Deterministic Pivot – Median of Three
    - Pivot chosen as the median of: First, Middle, Last elements.
    - More stable.
    - Predictable performance.

8. Program Features

The system provides:

1. Full Patient Display
   Shows the severity and triage level for each patient.
2. Statistics
   Displays:
    - Total patients
    - Number of patients per triage level
    - Average severity
3. Triage Help Menu
   Explains each triage level.
4. Pivot Comparison Tool
   Automatically compares:
    - Randomized QuickSort
    - Deterministic QuickSort
    - Randomized QuickSelect
    - Deterministic QuickSelect

The system displays average runtimes for each approach.

9. Conclusion

This project effectively shows how that application of algorithmic methods can aid in hospital emergency operations.
By implementing QuickSort and QuickSelect, we were able to create a system that effectively identifies the most urgent patients.

Through comparison of randomized and deterministic pivot strategies, we found that randomized pivot typically outperforms on large datasets, whereas deterministic pivot ensures stable performance.

The system satisfies all the requirements outlines in the CMPS441 Final Project, including real input, algorithm execution, sorting, selection, and runtime comparison.

10. Full Code

Patient Class:

```
public class Patient {
    private String name;
    private double severity;

    public Patient(String name, double severity) {
        this.name = name;
        this.severity = severity;
    }

    public String getName() {
        return name;
    }
```

```
    public double getSeverity() {
        return severity;
    }


    public String toString() {
        return name + " (severity = " + severity + ")";
    }
}
```

QuickAlgorithms Class:

```
import java.util.*;

public class QuickAlgorithms {

    private static Random rand = new Random();


    public static void quickSort(Patient[] arr, boolean randomized) {
        quickSort(arr, 0, arr.length - 1, randomized);
    }

    public static Patient quickSelect(Patient[] arr, int k, boolean randomized) {
        Patient[] copy = Arrays.copyOf(arr, arr.length);
        return quickSelectHelper(copy, 0, copy.length - 1, k - 1, randomized);
    }
```

```
private static void quickSort(Patient[] arr, int low, int high, boolean randomized) {
    if (low < high) {
        int p = partition(arr, low, high, randomized);
        quickSort(arr, low, p - 1, randomized);
        quickSort(arr, p + 1, high, randomized);
    }
}

private static Patient quickSelectHelper(Patient[] arr, int low, int high,
                                         int k, boolean randomized) {
    if (low == high) return arr[low];

    int p = partition(arr, low, high, randomized);
    int leftSize = p - low;

    if (k == leftSize)
        return arr[p];
    else if (k < leftSize)
        return quickSelectHelper(arr, low, p - 1, k, randomized);
    else
        return quickSelectHelper(arr, p + 1, high, k - leftSize - 1, randomized);
}

private static int partition(Patient[] arr, int low, int high, boolean randomized) {
    int pivotIndex = randomized
            ? low + rand.nextInt(high - low + 1)
            : medianOfThree(arr, low, high);

    swap(arr, pivotIndex, high);
```

```
    double pivot = arr[high].getSeverity();

    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j].getSeverity() > pivot) {
            i++;
            swap(arr, i, j);
        }
    }

    swap(arr, i + 1, high);
    return i + 1;
}

private static int medianOfThree(Patient[] arr, int low, int high) {
    int mid = (low + high) / 2;

    double a = arr[low].getSeverity();
    double b = arr[mid].getSeverity();
    double c = arr[high].getSeverity();

    if ((a <= b && b <= c) || (c <= b && b <= a)) return mid;
    if ((b <= a && a <= c) || (c <= a && a <= b)) return low;
    return high;
}

private static void swap(Patient[] arr, int i, int j) {
    Patient temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
```

```
    }
}
```

TriageSystem Class (main)

```
import java.util.*;

public class TriageSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Random rand = new Random();
        Patient[] patients = null;

        System.out.println("=== Hospital Emergency Triage System ===");


        while (patients == null) {
            System.out.println("\nSelect input method:");
            System.out.println("1. Manual patient entry");
            System.out.println("2. Randomly generate patients");
            System.out.println("3. Exit");
            System.out.print("> ");

            String choice = sc.nextLine();

            if (choice.equals("1")) {
                patients = manualInput(sc);
```

```
            break;
        }

        else if (choice.equals("2")) {
            System.out.print("How many patients? ");
            int n = Integer.parseInt(sc.nextLine());
            patients = randomPatients(n);
            System.out.println("Random patients generated.");
            break;
        }

        else if (choice.equals("3")) {
            System.out.println("Goodbye.");
            return;
        }

        else {
            System.out.println("Invalid choice.");
        }
    }


    while (true) {
        System.out.println("\nMain Menu");
        System.out.println("1. Show patients");
        System.out.println("2. Find k-th most critical (QuickSelect)");
        System.out.println("3. Sort patients by severity (QuickSort)");
        System.out.println("4. Compare algorithms");
        System.out.println("5. Search by name");
        System.out.println("6. Statistics");
```

```
        System.out.println("7. Triage help");
        System.out.println("8. Exit");
        System.out.print("> ");

        String op = sc.nextLine();

        switch (op) {

            case "1":
                printPatients(patients);
                break;

            case "2":
                doQuickSelect(sc, patients);
                break;

            case "3":
                doQuickSort(sc, patients);
                break;

            case "4":
                compareAlgorithms(patients);
                break;

            case "5":
                searchPatients(sc, patients);
                break;

            case "6":
                showStatistics(patients);
```

```
                break;

            case "7":
                showHelp();
                break;

            case "8":
                System.out.println("Goodbye.");
                return;

            default:
                System.out.println("Invalid option.");
                break;
        }
    }
}



private static Patient[] manualInput(Scanner sc) {
    System.out.print("How many patients? ");
    int n = Integer.parseInt(sc.nextLine());
    if (n <= 0) n = 1;

    Patient[] arr = new Patient[n];

    for (int i = 0; i < n; i++) {
        System.out.println("\nPatient #" + (i + 1));

        System.out.print("Name: ");
```

```
            String name = sc.nextLine();

            System.out.print("Severity (0-100): ");
            double s = Double.parseDouble(sc.nextLine());

            arr[i] = new Patient(name, s);
        }
        return arr;
    }

    private static Patient[] randomPatients(int n) {
        Random rand = new Random();
        Patient[] arr = new Patient[n];

        for (int i = 0; i < n; i++) {
            String name = "Patient_" + (i + 1);
            double s = Math.round(rand.nextDouble() * 10000.0) / 100.0;
            arr[i] = new Patient(name, s);
        }
        return arr;
    }


    private static void printPatients(Patient[] arr) {
        System.out.println("\n--- Patient List ---");
        for (Patient p : arr) {
            System.out.println(p);
        }
    }
```

```
private static void doQuickSelect(Scanner sc, Patient[] arr) {
    System.out.print("Enter k: ");
    int k = Integer.parseInt(sc.nextLine());

    System.out.println("1. Random pivot");
    System.out.println("2. Deterministic pivot");
    System.out.print("> ");
    boolean rnd = sc.nextLine().equals("1");

    Patient p = QuickAlgorithms.quickSelect(arr, k, rnd);

    System.out.println("\nK-th most critical patient:");
    System.out.println(p);
}

private static void doQuickSort(Scanner sc, Patient[] arr) {
    System.out.println("1. Random pivot");
    System.out.println("2. Deterministic pivot");
    System.out.print("> ");
    boolean rnd = sc.nextLine().equals("1");

    Patient[] sorted = Arrays.copyOf(arr, arr.length);
    QuickAlgorithms.quickSort(sorted, rnd);

    System.out.println("\n--- Sorted Patients ---");
    for (Patient p : sorted) {
        System.out.println(p);
    }
}
```

```
private static void searchPatients(Scanner sc, Patient[] arr) {
    System.out.print("Enter name keyword: ");
    String key = sc.nextLine().toLowerCase();

    boolean found = false;
    for (Patient p : arr) {
        if (p.getName().toLowerCase().contains(key)) {
            System.out.println(p);
            found = true;
        }
    }
    if (!found)
        System.out.println("No matching patients.");
}

private static void showStatistics(Patient[] arr) {
    double sum = 0;
    for (Patient p : arr) sum += p.getSeverity();

    System.out.println("\nTotal patients: " + arr.length);
    System.out.println("Average severity: " + (sum / arr.length));
}

private static void showHelp() {
    System.out.println("\n--- Triage Help ---");
    System.out.println("Severity ≥ 75: Critical");
    System.out.println("50–74: High Priority");
    System.out.println("25–49: Normal");
    System.out.println("<25: Minor");
}
```

```
    private static void compareAlgorithms(Patient[] arr) {
        long start, end;

        start = System.nanoTime();
        QuickAlgorithms.quickSelect(arr, 1, true);
        end = System.nanoTime();
        System.out.println("QuickSelect Random: " + (end - start) + " ms");

        start = System.nanoTime();
        QuickAlgorithms.quickSelect(arr, 1, false);
        end = System.nanoTime();
        System.out.println("QuickSelect Deterministic: " + (end - start) + " ms");
    }
}
```

11. Sample Output

```
=== Hospital Emergency Triage System ===

Select input method:
1. Manual patient entry
2. Randomly generate patients
3. Exit
>
```

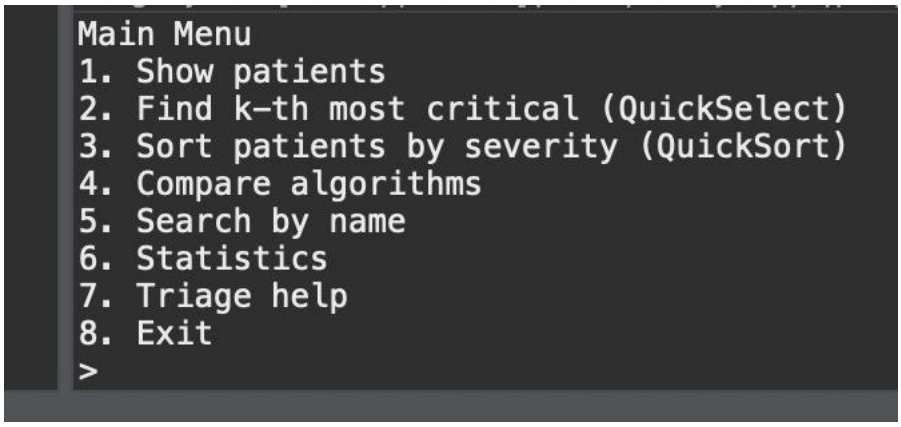
This is the main menu of the Emergency Triage System. The user gets to choose between 2 input methods, either to manually enter patient or let the system randomly generate patient.

```
Select input method:
1. Manual patient entry
2. Randomly generate patients
3. Exit
> 1
How many patients? 2

Patient #1
Name: Nour
Severity (0-100): 40

Patient #2
Name: Sadeq
Severity (0-100): 84
```

If the user chooses to manually enter the patients, the system will ask them how many patients is there followed by their name and severity score (meaning how fast do they need assistance).



Then the system will show this to the user.

In the main menu there are 8 options the user can choose from, either to view the patient list, find the k-th most critical patient using QuickSelect, sort patient by severity with Quicksort, compare algorithm performance, search for a patient by name, view system statistics, access triage help or exit the application.

```
> 1

--- Patient List ---
1. Nour | Severity: 40.0 | Level 3 (Normal — can wait)
2. Sadeq | Severity: 84.0 | Level 1 (Critical — immediate treatment)
```

Here the user chose number 1 which will list all the patient with their severity score.

```
> 2
Enter k: 1
Choose pivot strategy:
1. Randomized pivot
2. Deterministic pivot (Median-of-Three)
> 1

Result: Sadeq
Severity: 84.0
Triage Level 1 (Critical — immediate treatment)
Time: 3.497917 ms
```

If the user chooses 2 it will ask it to enter k, which means you write the number you want based on the most critical patient you want. Then it asks you to enter what pivot strategy you want. The system finally prints the result.

```
Choose pivot strategy:
1. Randomized pivot
2. Deterministic pivot (Median-of-Three)
> 2

--- Patient List ---
1. Sadeq | Severity: 84.0 | Level 1 (Critical — immediate treatment)
2. Nour | Severity: 40.0 | Level 3 (Normal — can wait)

Sort time: 0.08275 ms
```

Here if the user chooses 3 it will ask it to the strategy they want, Then the system finally prints the sorted list based on the severity case.

```
> 4

=== Average Times (ms) ===
QuickSelect (random):      0.011016799999999998
QuickSelect (deterministic): 0.0048336
QuickSort (random):        0.003325
QuickSort (deterministic): 0.0025998
```

This output evaluates the time taken by QuickSelect and QuickSort when applying a randomized pivot compared to a deterministic pivot.
The results reveal that the randomized pivot often executes faster, while the deterministic pivot offers stable results.

```
Main Menu
1. Show patients
2. Find k-th most critical patient (QuickSelect)
3. Sort patients by severity (QuickSort)
4. Compare randomized vs deterministic performance
5. Search patient by name
6. Show statistics
7. Triage help
8. Exit
> 5
Enter name keyword: sadeq
Sadeq | Severity: 84.0 | Level 1 (Critical — immediate treatment)
```

Here the user chooses 5 where they system will ask the user to enter the patient's name and then the system will print the patient severity level.

```
> 6

--- Statistics ---
Total patients: 2
Critical (1): 1
High (2): 0
Normal (3): 1
Minor (4): 0
Average severity: 62,00
```

In option 6, the system will print the statistics of the patients in every level and the average severity score.

```
8. Exit
> 7

--- Triage Help ---
1 = Critical (severity ≥ 75)
2 = High Priority (severity ≥ 50)
3 = Normal Priority (severity ≥ 25)
4 = Minor (severity < 25)
```

Option 7, they system will print the severity score for each level.

```
Main Menu
1. Show patients
2. Find k-th most critical (QuickSelect)
3. Sort patients by severity (QuickSort)
4. Compare algorithms
5. Search by name
6. Statistics
7. Triage help
8. Exit
> 8
Goodbye.
```

Option 8, the user exits the system.